Computer Networks Lab Record: Experiments 1-13

Oct 13, 2026 · @Roshin R G

Experiment 1: Network Commands in Linux and Windows

Aim

To study and execute the common network commands used in Linux and Windows and observe their output.

Algorithm

1. Open a terminal (Linux) or Command Prompt (Windows).
2. Type the command with its options and press Enter.
3. Observe the output and note what each field means.
4. Repeat for every command in the table below.

Commands

Command	Linux	Windows	Purpose
Show IP configuration	`ifconfig` or `ip addr`	`ipconfig /all`	Shows IP, mask, gateway, MAC, DNS
Test connectivity	`ping -c 4 google.com`	`ping -n 4 google.com`	Sends ICMP echo requests, reports RTT and loss
Trace route	`traceroute google.com`	`tracert google.com`	Lists each router hop to the destination
DNS lookup	`nslookup google.com` or `dig google.com`	`nslookup google.com`	Resolves a domain name to an IP
Connections and ports	`netstat -tuln` or `ss -tuln`	`netstat -an`	Lists open ports and active connections
ARP cache	`arp -a`	`arp -a`	Shows IP to MAC mappings

Command	Linux	Windows	Purpose
Routing table	`route -n` or `ip route`	`route print`	Shows the kernel routing table
Host name	`hostname`	`hostname`	Displays the system name
Interface statistics	`ip -s link`	`netstat -e`	Packets sent, received, errors
Flush DNS cache	`resolvectl flush-caches`	`ipconfig /flushdns`	Clears cached DNS entries
Release/renew DHCP	`sudo dhclient -r; sudo dhclient`	`ipconfig /release` then `ipconfig /renew`	Gets a new lease
Path statistics	`mtr google.com`	`pathping google.com`	Combines ping and traceroute
Whois	`whois example.com`	(not built in)	Registration data of a domain

Program (commands executed)

```
# Linux
ifconfig
ping -c 4 google.com
traceroute google.com
nslookup google.com
netstat -tuln
arp -a
route -n
hostname
```

```
REM Windows
ipconfig /all
ping -n 4 google.com
tracert google.com
nslookup google.com
netstat -an
arp -a
route print
hostname
```

Output (sample)

```
$ ping -c 4 google.com
PING google.com (142.250.195.78) 56(84) bytes of data.
64 bytes from maa03s43-in-f14.1e100.net: icmp_seq=1 ttl=117 time=14.2 ms
64 bytes from maa03s43-in-f14.1e100.net: icmp_seq=2 ttl=117 time=13.8 ms
64 bytes from maa03s43-in-f14.1e100.net: icmp_seq=3 ttl=117 time=14.0 ms
64 bytes from maa03s43-in-f14.1e100.net: icmp_seq=4 ttl=117 time=13.9 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 13.8/14.0/14.2/0.15 ms

$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
      inet 192.168.1.10  netmask 255.255.255.0  broadcast 192.168.1.255
      ether 08:00:27:4e:66:a1  txqueuelen 1000

C:\> ipconfig
Ethernet adapter Ethernet:
   IPv4 Address. . . . . . . . . . . : 192.168.1.10
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1

C:\> tracert google.com
  1    <1 ms    <1 ms    <1 ms  192.168.1.1
  2     4 ms     3 ms     4 ms  10.10.0.1
  3    13 ms    14 ms    13 ms  142.250.195.78
Trace complete.
```

IP addresses, times and MACs will differ on your machine; copy your own output into the record.

Result

The Linux and Windows network commands were executed and their output was studied successfully.

Experiment 2: Hub and Switch Networks in Cisco Packet Tracer

Aim

To design simple networks using only a hub, only a switch, and both, in Cisco Packet Tracer, and to find the functional difference between a hub and a switch.

Algorithm / Procedure

Part A: Hub-only network (4 to 6 hosts)

1. Open Packet Tracer. From End Devices, drag 4 PCs (PC0 to PC3) to the workspace.
2. From Network Devices > Hubs, drag one Generic Hub (Hub0).
3. Select Copper Straight-Through cable and connect each PC's FastEthernet0 to a hub port.
4. Click each PC > Desktop > IP Configuration and set: PC0 192.168.1.1, PC1 192.168.1.2, PC2 192.168.1.3, PC3 192.168.1.4, mask 255.255.255.0.
5. Switch to Simulation mode. Add a simple PDU (envelope icon) from PC0 to PC2.
6. Click Capture/Forward repeatedly and watch the packet path.

Part B: Switch-only network

1. Replace the hub with a 2960 switch (Switch0) and connect the same 4 PCs with straight-through cables.
2. Keep the same IP addresses.
3. In Simulation mode send a simple PDU from PC0 to PC2 twice, using Capture/Forward, and observe.

Part C: Hub and switch together

1. Place one switch and one hub. Connect Switch0 to Hub0 using a Copper Cross-Over cable.
2. Attach PC0 and PC1 to the switch, and PC2 and PC3 to the hub. Assign 192.168.1.1 to .4.
3. Send PDUs between PCs on different devices and observe the broadcast behavior.

Part D: College topology

1. Study the college network (usually a star or extended star: PCs to access switches, access switches to a core/distribution switch, core to router/firewall and ISP).
2. Draw and label it in the observation book.

Program (device configuration)

No code is needed; devices are configured through the GUI. Verification command on a PC (Desktop > Command Prompt):

```
PC> ping 192.168.1.3
```

On the switch (CLI) to view the learned MAC table:

```
Switch> enable
Switch# show mac address-table
```

Output

```
PC> ping 192.168.1.3
Pinging 192.168.1.3 with 32 bytes of data:
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

Switch# show mac address-table
 Vlan    Mac Address       Type      Ports
 ----    -----------       ----      -----
    1    0001.4321.a1b2    DYNAMIC   Fa0/1
    1    0002.1743.c3d4    DYNAMIC   Fa0/2
    1    0003.e45a.f5a6    DYNAMIC   Fa0/3
    1    0004.9a5b.1c7d    DYNAMIC   Fa0/4
```

Simulation observations:

Device	Behavior seen in Simulation mode
Hub	A frame from PC0 is copied to every other port, so PC1, PC2 and PC3 all receive it; only PC2 accepts it. One collision domain.

Device	Behavior seen in Simulation mode
Switch	First frame is flooded (MAC not yet learned). After the table is learned, the frame goes only to the destination port. One collision domain per port.
Hub + Switch	Frames crossing the hub are repeated to all hub ports; the switch forwards only when the destination MAC is on another segment.

Result

Networks with a hub, a switch, and both were built and simulated. The hub is a Layer 1 device that broadcasts every frame to all ports, while the switch is a Layer 2 device that learns MAC addresses and forwards frames only to the intended port.

Experiment 3: Setting up a LAN with a Switch

Aim

To set up and configure a Local Area Network using a switch and Ethernet cables, assign IP addresses, test connectivity with ping, and share files across the hosts.

Algorithm / Procedure

1. Keep 3 or 4 computers, one switch, and straight-through (CAT5/CAT6) Ethernet cables ready.
2. Connect each computer's NIC to a switch port; check that the link LEDs glow.
3. Assign IP addresses (all in the same network, mask 255.255.255.0):
   - Windows: Control Panel > Network and Sharing Centre > Change adapter settings > Ethernet > Properties > Internet Protocol Version 4 > Use the following IP address.
   - Linux: `sudo ip addr add 192.168.10.1/24 dev eth0` (or set it in Network settings).
4. Check connectivity: from each host run `ping <IP of another host>`.
5. Share a folder: on Host 1 right-click a folder > Properties > Sharing > Share, give Everyone read/write. (Linux: configure Samba and add a share in `/etc/samba/smb.conf`.)
6. From Host 2 open File Explorer and type `\\192.168.10.1` in the address bar, open the shared folder, copy a file in and out.

Program (configuration and commands)

Host	IP address	Subnet mask
Host 1	192.168.10.1	255.255.255.0
Host 2	192.168.10.2	255.255.255.0
Host 3	192.168.10.3	255.255.255.0
Host 4	192.168.10.4	255.255.255.0

```
ping 192.168.10.2
ping 192.168.10.3
net use Z: \\192.168.10.1\Shared
```

Output

```
C:\> ping 192.168.10.2
Pinging 192.168.10.2 with 32 bytes of data:
Reply from 192.168.10.2: bytes=32 time=1ms TTL=128
Reply from 192.168.10.2: bytes=32 time<1ms TTL=128
Reply from 192.168.10.2: bytes=32 time<1ms TTL=128
Reply from 192.168.10.2: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.10.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

The shared folder opened from Host 2 and a test file was copied successfully in both directions.

Result

A LAN was built with a switch and Ethernet cables, IP addresses were assigned, connectivity was verified with ping, and files were shared across the hosts.

Experiment 4: Error Detection and Correction using Hamming Code

Aim

To write a program that implements error detection and correction using the Hamming code (7,4) and verify the correction with a test run.

Algorithm

Sender

1. Read the 4 data bits d1 d2 d3 d4.
2. Place them at positions 3, 5, 6, 7 of a 7-bit codeword; positions 1, 2, 4 are the parity bits p1, p2, p4.
3. Compute even parity: p1 = d1 xor d2 xor d4 (positions 1,3,5,7); p2 = d1 xor d3 xor d4 (positions 2,3,6,7); p4 = d2 xor d3 xor d4 (positions 4,5,6,7).
4. Transmit the 7-bit codeword.

Receiver

1. Read the received 7 bits.
2. Recompute the three parity checks c1 (positions 1,3,5,7), c2 (positions 2,3,6,7), c4 (positions 4,5,6,7).
3. Error position = c4 c2 c1 as a binary number. If 0, there is no error.
4. If non-zero, flip the bit at that position to correct it.
5. Extract data bits from positions 3, 5, 6, 7.

Program (Python)

```
def encode(d):
    # d = [d1, d2, d3, d4]; codeword positions 1..7 (index 0..6)
    c = [0] * 7
    c[2], c[4], c[5], c[6] = d[0], d[1], d[2], d[3]
    c[0] = c[2] ^ c[4] ^ c[6]          # p1 -> positions 1,3,5,7
    c[1] = c[2] ^ c[5] ^ c[6]          # p2 -> positions 2,3,6,7
    c[3] = c[4] ^ c[5] ^ c[6]          # p4 -> positions 4,5,6,7
    return c

def decode(r):
    c1 = r[0] ^ r[2] ^ r[4] ^ r[6]
    c2 = r[1] ^ r[2] ^ r[5] ^ r[6]
    c4 = r[3] ^ r[4] ^ r[5] ^ r[6]
    pos = c4 * 4 + c2 * 2 + c1
    if pos != 0:
        print("Error detected at position", pos)
        r[pos - 1] ^= 1               # correct the bit
    else:
        print("No error detected")
    return r, [r[2], r[4], r[5], r[6]]
```

```
data = list(map(int, input("Enter 4 data bits (e.g. 1011): ").strip()))
code = encode(data)
print("Transmitted codeword :", "".join(map(str, code)))

recv = list(map(int, input("Enter received 7-bit word: ").strip()))
corrected, original = decode(recv)
print("Corrected codeword  :", "".join(map(str, corrected)))
print("Recovered data bits :", "".join(map(str, original)))
```

Output

```
Enter 4 data bits (e.g. 1011): 1011
Transmitted codeword : 0110011
Enter received 7-bit word: 0110111
Error detected at position 5
Corrected codeword  : 0110011
Recovered data bits : 1011
```

Check by hand: data 1011 gives p1 = 1^0^1 = 0, p2 = 1^1^1 = 1, p4 = 0^1^1 = 0, so the codeword is p1 p2 d1 p4 d2 d3 d4 = 0 1 1 0 0 1 1. Bit 5 was flipped in the received word and the decoder found it.

Result

The Hamming (7,4) program detected and corrected a single-bit error and recovered the original data.

Experiment 5: Flow Control using Sliding Window Protocol

Aim

To write a program that implements flow control at the data link layer using the sliding window protocol and simulates the flow of frames from one node to another.

Algorithm

1. Read the total number of frames and the window size.
2. Sender transmits frames from the current window (base to base + window size - 1).
3. Receiver acknowledges each frame it receives in order.
4. When the acknowledgement for the first frame in the window arrives, the window slides forward by one (or by the number of frames acknowledged).

5. Repeat until all frames are sent and acknowledged.
6. Stop.

Program (Python)

```
total = int(input("Enter total number of frames: "))
w = int(input("Enter window size: "))

base = 0                      # first unacknowledged frame
next_frame = 0                # next frame to send

print()
while base < total:
    # sender: send every frame inside the window
    while next_frame < base + w and next_frame < total:
        print(f"Sender   : sent frame {next_frame}")
        next_frame += 1

    # receiver: acknowledge the frames that were sent
    for f in range(base, next_frame):
        print(f"Receiver : received frame {f}, sent ACK {f}")

    print(f"Sender   : ACKs received, window slides from {base} to
{next_frame}\n")
    base = next_frame

print("All frames sent and acknowledged.")
```

Output

```
Enter total number of frames: 7
Enter window size: 3

Sender   : sent frame 0
Sender   : sent frame 1
Sender   : sent frame 2
Receiver : received frame 0, sent ACK 0
Receiver : received frame 1, sent ACK 1
Receiver : received frame 2, sent ACK 2
Sender   : ACKs received, window slides from 0 to 3

Sender   : sent frame 3
Sender   : sent frame 4
```

```
Sender   : sent frame 5
Receiver : received frame 3, sent ACK 3
Receiver : received frame 4, sent ACK 4
Receiver : received frame 5, sent ACK 5
Sender   : ACKs received, window slides from 3 to 6

Sender   : sent frame 6
Receiver : received frame 6, sent ACK 6
Sender   : ACKs received, window slides from 6 to 7

All frames sent and acknowledged.
```

Result

The sliding window protocol was simulated: at most `w` frames were outstanding at a time, and the window advanced as acknowledgements arrived.

Experiment 6: VLAN and Wireless LAN in Cisco Packet Tracer

6a. Virtual LAN configuration

Aim

To simulate a Virtual LAN (VLAN) configuration using Cisco Packet Tracer.

Algorithm / Procedure

1. Place one 2960 switch and 4 PCs. Connect PC0 and PC1 to Fa0/1 and Fa0/2, PC2 and PC3 to Fa0/3 and Fa0/4.
2. Assign IPs: PC0 192.168.1.1, PC1 192.168.1.2, PC2 192.168.1.3, PC3 192.168.1.4 (mask 255.255.255.0).
3. Open the switch CLI and create VLAN 10 and VLAN 20.
4. Assign Fa0/1 and Fa0/2 to VLAN 10; Fa0/3 and Fa0/4 to VLAN 20.
5. Verify with `show vlan brief`; ping within a VLAN and across VLANs.

Program (switch CLI)

```
Switch> enable
Switch# configure terminal
Switch(config)# vlan 10
Switch(config-vlan)# name FACULTY
Switch(config-vlan)# exit
Switch(config)# vlan 20
Switch(config-vlan)# name STUDENTS
Switch(config-vlan)# exit
Switch(config)# interface range fastEthernet 0/1 - 2
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport access vlan 10
Switch(config-if-range)# exit
Switch(config)# interface range fastEthernet 0/3 - 4
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport access vlan 20
Switch(config-if-range)# end
Switch# show vlan brief
```

Output

```
VLAN Name               Status    Ports
---- ------------------ --------- -------------------------------
1    default            active    Fa0/5, Fa0/6, ... Gig0/2
10   FACULTY            active    Fa0/1, Fa0/2
20   STUDENTS           active    Fa0/3, Fa0/4

PC0> ping 192.168.1.2     -> Reply (4/4 received)
PC0> ping 192.168.1.3     -> Request timed out (0/4 received)
```

Result

Two VLANs were created. Hosts in the same VLAN communicate; hosts in different VLANs cannot, without a router.

6b. VLAN for Robotics department faculty in 3 blocks

Aim

To configure a VLAN so that 10 Robotics faculty sitting in 3 different blocks are logically in the same LAN.

Algorithm / Procedure

1. Use one switch per block (Switch-A, Switch-B, Switch-C) and connect them to each other with trunk links (Switch-A to Switch-B, Switch-B to Switch-C).
2. Place the faculty PCs: 4 in Block A, 3 in Block B, 3 in Block C. Give them 192.168.50.1 to 192.168.50.10 with mask 255.255.255.0.
3. On every switch create VLAN 50 named ROBOTICS and put the faculty ports in it as access ports.
4. Configure the inter-switch ports as trunk ports so VLAN 50 traffic crosses blocks.
5. Ping from a PC in Block A to a PC in Block C.

Program (same on each switch; trunk ports differ)

```
Switch(config)# vlan 50
Switch(config-vlan)# name ROBOTICS
Switch(config-vlan)# exit
Switch(config)# interface range fastEthernet 0/1 - 4
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport access vlan 50
Switch(config-if-range)# exit
Switch(config)# interface gigabitEthernet 0/1
Switch(config-if)# switchport mode trunk
Switch(config-if)# end
Switch# show interfaces trunk
```

Output

```
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

PC-A1> ping 192.168.50.9
Reply from 192.168.50.9: bytes=32 time=2ms TTL=128   (4/4 received)
```

Result

All 10 faculty, though in three blocks, behave as one logical LAN through VLAN 50 and trunk links.

6c. Wireless LAN

Aim

To configure a Wireless LAN using Cisco Packet Tracer.

Algorithm / Procedure

1. Place a Wireless Router (e.g. WRT300N) and 2 laptops and 1 smartphone.
2. For laptops, power off, replace the wired NIC with the WMP300N wireless module, power on.
3. Click the wireless router > Config > Wireless: set SSID `LabWiFi`, security WPA2-PSK, passphrase `lab12345`.
4. Router GUI > Setup: LAN IP 192.168.0.1, DHCP enabled.
5. On each laptop, PC Wireless > Connect: choose LabWiFi and enter the passphrase.
6. Check the IP assigned by DHCP and ping between the wireless devices.

Program

Configured through the router's GUI (SSID, security, DHCP). Test on laptop Command Prompt:

```
PC> ipconfig
PC> ping 192.168.0.102
```

Output

```
IP Address......: 192.168.0.101
Subnet Mask.....: 255.255.255.0
Default Gateway.: 192.168.0.1

PC> ping 192.168.0.102
Reply from 192.168.0.102: bytes=32 time=12ms TTL=128   (4/4 received)
```

Result

The wireless LAN was configured; devices associated with the SSID, obtained addresses by DHCP, and communicated successfully.

Experiment 7: Subnetting in Cisco Packet Tracer

Aim

To implement subnetting, design multiple subnets with a suitable number of hosts, assign static IPs, connect the subnets through a router, and observe packet transmission with and without the router.

Algorithm / Procedure

1. Take the network 192.168.1.0/24 and divide it into 2 subnets. Borrow 1 host bit, so the mask becomes /25 (255.255.255.128), giving 126 usable hosts per subnet.
2. Subnet table:

Subnet	Network ID	Usable range	Broadcast	Mask
A	192.168.1.0	192.168.1.1 to 192.168.1.126	192.168.1.127	255.255.255.128
B	192.168.1.128	192.168.1.129 to 192.168.1.254	192.168.1.255	255.255.255.128

3. Place 2 switches, 1 router (2811 or 1941) and 2 PCs per subnet. Connect PCs to their switch, and each switch to a router interface.
4. Assign static IPs: PC0 192.168.1.2, PC1 192.168.1.3 (gateway 192.168.1.1); PC2 192.168.1.130, PC3 192.168.1.131 (gateway 192.168.1.129).
5. Configure the router interfaces as the gateways (CLI below).
6. *With router:* ping from PC0 to PC2 and trace in Simulation mode.
7. *Without router:* remove the router (or shut its interfaces), ping again from PC0 to PC2 and observe.

Program (router CLI)

```
Router> enable
Router# configure terminal
Router(config)# interface gigabitEthernet 0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.128
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# interface gigabitEthernet 0/1
Router(config-if)# ip address 192.168.1.129 255.255.255.128
Router(config-if)# no shutdown
Router(config-if)# end
Router# show ip route
```

Output

```
Router# show ip route
C    192.168.1.0/25 is directly connected, GigabitEthernet0/0
C    192.168.1.128/25 is directly connected, GigabitEthernet0/1

With router:
PC0> ping 192.168.1.130
Reply from 192.168.1.130: bytes=32 time=2ms TTL=127   (4/4 received)

Without router:
PC0> ping 192.168.1.130
Request timed out.   (0/4 received, packet dropped: different subnet, no
gateway)
```

Result

The network was divided into two subnets. Hosts in different subnets communicated only when connected through a router; without the router the packets were dropped.

Experiment 8: Internetworking with Routers

8a. Simple internetwork using a router

Aim

To design and configure a simple internetwork with a router, assign static IPs, and trace how routing happens in packet transmission.

Algorithm / Procedure

1. Place one router, two switches and 3 PCs per network (6 PCs in all).
2. Network 1: 192.168.10.0/24 on Router Gi0/0 (192.168.10.1). Network 2: 192.168.20.0/24 on Router Gi0/1 (192.168.20.1).
3. Give PCs static IPs: Network 1 hosts 192.168.10.2 to .4 with gateway 192.168.10.1; Network 2 hosts 192.168.20.2 to .4 with gateway 192.168.20.1.
4. Configure the router interfaces and bring them up.
5. In Simulation mode send a PDU from 192.168.10.2 to 192.168.20.3 and follow it: PC > switch > router (ARP, route lookup) > switch > PC.

Program (router CLI)

```
Router(config)# interface gigabitEthernet 0/0
Router(config-if)# ip address 192.168.10.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# interface gigabitEthernet 0/1
Router(config-if)# ip address 192.168.20.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# end
Router# show ip route
```

Output

```
C  192.168.10.0/24 is directly connected, GigabitEthernet0/0
C  192.168.20.0/24 is directly connected, GigabitEthernet0/1

PC> ping 192.168.20.3
Reply from 192.168.20.3: bytes=32 time=1ms TTL=127   (4/4 received)
```

The first ping may show one timeout while ARP resolves; the TTL of 127 shows one router hop was crossed.

Result

Two networks were interconnected using a router and packets were routed between them.

8b. Wireless router, DHCP server and Internet cloud

Aim

To design an internetwork using a wireless router, a DHCP server and an Internet cloud.

Algorithm / Procedure

1. Place a Wireless Router, 2 laptops, a PC, a Server (DHCP/Web) and a Cloud-PT (Internet).
2. Connect the wireless router's Internet port to the cloud with a straight-through cable; connect the server to the router LAN port.
3. On the server: Services > DHCP, turn on, set default gateway 192.168.0.1, DNS 8.8.8.8, start IP 192.168.0.10, mask 255.255.255.0.
4. On the router set the wireless SSID and security and switch its own DHCP off if the server will serve addresses.

5. Set the clients to DHCP; connect the laptops to the SSID.
6. Verify the leased addresses and ping the server and the remote web server.

Program

GUI based; verification on a client:

```
PC> ipconfig /renew
PC> ipconfig
PC> ping 192.168.0.1
```

Output

```
IP Address......: 192.168.0.10
Subnet Mask.....: 255.255.255.0
Default Gateway.: 192.168.0.1
DNS Server......: 8.8.8.8
PC> ping 192.168.0.1   -> Reply (4/4 received)
```

Result

Clients obtained addresses automatically from the DHCP server and reached the router and the Internet cloud.

8c. Inter-network using switch, router and Ethernet cables (lab)

Aim

To build a physical inter-network in the lab using a switch, a router and Ethernet cables.

Algorithm / Procedure

1. Connect 2 PCs to Switch 1 and 2 PCs to Switch 2; connect each switch to a different router port with straight-through cables (use a console cable for router setup).
2. Assign IPs and gateways as in 8a.
3. Configure the router interfaces over the console (same commands as 8a).
4. Test with ping and tracert across the networks.

Output

```
C:\> tracert 192.168.20.3
  1    <1 ms    <1 ms    <1 ms  192.168.10.1
  2     1 ms    <1 ms    <1 ms  192.168.20.3
Trace complete.
```

Result

The lab inter-network was built and hosts on different networks communicated through the router.

Experiment 9: Static Routing and RIP

9a. Static routing

Aim

To simulate static routing configuration using Cisco Packet Tracer.

Algorithm / Procedure

1. Place two routers (R1, R2), connect them with a serial DCE/DTE link (or Gi-to-Gi with a cross-over cable). Attach one switch and one PC to each router's LAN.
2. Addressing: LAN1 192.168.1.0/24 (R1 Gi0/0 = .1, PC1 = .2); link 10.0.0.0/30 (R1 = 10.0.0.1, R2 = 10.0.0.2); LAN2 192.168.2.0/24 (R2 Gi0/0 = .1, PC2 = .2).
3. Configure interfaces on both routers.
4. Add static routes: R1 needs a route to 192.168.2.0 via 10.0.0.2; R2 needs a route to 192.168.1.0 via 10.0.0.1.
5. Verify with `show ip route` and ping PC1 to PC2.

Program

```
! R1
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config)# interface gigabitEthernet 0/1
R1(config-if)# ip address 10.0.0.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# ip route 192.168.2.0 255.255.255.0 10.0.0.2

! R2
R2(config)# interface gigabitEthernet 0/0
R2(config-if)# ip address 192.168.2.1 255.255.255.0
R2(config-if)# no shutdown
R2(config)# interface gigabitEthernet 0/1
R2(config-if)# ip address 10.0.0.2 255.255.255.252
R2(config-if)# no shutdown
R2(config-if)# exit
R2(config)# ip route 192.168.1.0 255.255.255.0 10.0.0.1
```

Output

```
R1# show ip route
C    10.0.0.0/30 is directly connected, GigabitEthernet0/1
C    192.168.1.0/24 is directly connected, GigabitEthernet0/0
S    192.168.2.0/24 [1/0] via 10.0.0.2

PC1> ping 192.168.2.2
Reply from 192.168.2.2: bytes=32 time=2ms TTL=126   (4/4 received)
```

Result

Static routes were configured and the two LANs communicated across the routers.

9b. RIP

Aim

To simulate the Routing Information Protocol (RIP) using Cisco Packet Tracer.

Algorithm / Procedure

1. Use the same topology and addressing as 9a; remove the static routes (`no ip route ...`).

2. On each router enable RIP version 2 and advertise the directly connected networks.
3. Wait about 30 seconds for updates; check the routing table for `R` entries.
4. Ping PC1 to PC2.

Program

```
R1(config)# router rip
R1(config-router)# version 2
R1(config-router)# no auto-summary
R1(config-router)# network 192.168.1.0
R1(config-router)# network 10.0.0.0
R1(config-router)# end

R2(config)# router rip
R2(config-router)# version 2
R2(config-router)# no auto-summary
R2(config-router)# network 192.168.2.0
R2(config-router)# network 10.0.0.0
R2(config-router)# end

R1# show ip route
R1# show ip protocols
```

Output

```
R1# show ip route
C    10.0.0.0/30 is directly connected, GigabitEthernet0/1
C    192.168.1.0/24 is directly connected, GigabitEthernet0/0
R    192.168.2.0/24 [120/1] via 10.0.0.2, 00:00:12, GigabitEthernet0/1

PC1> ping 192.168.2.2
Reply from 192.168.2.2: bytes=32 time=2ms TTL=126   (4/4 received)
```

Result

RIP was configured; routers learned each other's networks dynamically (administrative distance 120, hop count 1) and the LANs communicated.

Experiment 10: End-to-End Communication at the Transport Layer

Run the server in one terminal first, then the client in another. All programs use Python 3 and `localhost` (127.0.0.1).

10a-i. TCP echo client and server

Aim

To implement an echo client and server using TCP sockets.

Algorithm

Server: create a TCP socket, bind to an address and port, listen, accept a connection, receive data, send the same data back, repeat until the client closes, then close the socket.

Client: create a TCP socket, connect to the server, read a message from the user, send it, print the echoed reply, repeat until the user types `bye`, then close the socket.

Program: `tcp_echo_server.py`

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)
print("TCP echo server waiting on port 5000...")

conn, addr = server.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Received:", data.decode())
    conn.sendall(data)              # echo back
conn.close()
server.close()
```

Program: `tcp_echo_client.py`

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))

while True:
    msg = input("Enter message (bye to quit): ")
    client.sendall(msg.encode())
    if msg.lower() == "bye":
        break
    reply = client.recv(1024)
    print("Echo from server:", reply.decode())
client.close()
```

Output

```
# Server terminal
TCP echo server waiting on port 5000...
Connected by ('127.0.0.1', 51344)
Received: hello
Received: network lab
Received: bye

# Client terminal
Enter message (bye to quit): hello
Echo from server: hello
Enter message (bye to quit): network lab
Echo from server: network lab
Enter message (bye to quit): bye
```

Result

The TCP echo client and server were implemented; the server returned every message exactly as received.

10a-ii. UDP echo client and server

Aim

To implement an echo client and server using UDP sockets.

Algorithm

Server: create a UDP socket (`SOCK_DGRAM`), bind to a port, loop: `recvfrom` a datagram and its sender address, `sendto` the same data back to that address.

Client: create a UDP socket, `sendto` the message to the server address, `recvfrom` the reply, print it, repeat until `bye` , then close. No connection is established.

Program: `udp_echo_server.py`

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 6000))
print("UDP echo server waiting on port 6000...")

while True:
    data, addr = server.recvfrom(1024)
    print("Received from", addr, ":", data.decode())
    server.sendto(data, addr)
```

Program: `udp_echo_client.py`

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_addr = ("127.0.0.1", 6000)

while True:
    msg = input("Enter message (bye to quit): ")
    if msg.lower() == "bye":
        break
    client.sendto(msg.encode(), server_addr)
    reply, _ = client.recvfrom(1024)
    print("Echo from server:", reply.decode())
client.close()
```

Output

```
# Server terminal
UDP echo server waiting on port 6000...
Received from ('127.0.0.1', 49821) : hello udp

# Client terminal
Enter message (bye to quit): hello udp
Echo from server: hello udp
Enter message (bye to quit): bye
```

Result

The UDP echo client and server were implemented and tested.

10b. Chat program using sockets (TCP)

Aim

To implement a two-way chat program using socket programming.

Algorithm

Server: create a TCP socket, bind, listen, accept the client. Use a receiving thread that prints incoming messages while the main thread reads the user's input and sends it. Stop when either side sends `bye`.

Client: create a socket, connect to the server, then use the same two-way send and receive logic.

Program: `chat_server.py`

```
import socket, threading

def receive(conn):
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == "bye":
            print("\nClient left the chat.")
            break
        print("\nClient:", data)

s = socket.socket()
s.bind(("127.0.0.1", 7000))
s.listen(1)
print("Chat server waiting for a client...")
```

```
conn, addr = s.accept()
print("Client connected:", addr)

threading.Thread(target=receive, args=(conn,), daemon=True).start()
while True:
    msg = input()
    conn.sendall(msg.encode())
    if msg.lower() == "bye":
        break
conn.close()
s.close()
```

Program: `chat_client.py`

```
import socket, threading

def receive(sock):
    while True:
        data = sock.recv(1024).decode()
        if not data or data.lower() == "bye":
            print("\nServer left the chat.")
            break
        print("\nServer:", data)

c = socket.socket()
c.connect(("127.0.0.1", 7000))
print("Connected to chat server. Type bye to exit.")

threading.Thread(target=receive, args=(c,), daemon=True).start()
while True:
    msg = input()
    c.sendall(msg.encode())
    if msg.lower() == "bye":
        break
c.close()
```

Output

```
# Server terminal
Chat server waiting for a client...
Client connected: ('127.0.0.1', 52110)

Client: hi
hello, how are you?

Client: fine, bye
bye

# Client terminal
Connected to chat server. Type bye to exit.
hi

Server: hello, how are you?
fine, bye

Server: bye
```

Result

A two-way chat between a TCP client and server was implemented; messages were exchanged in both directions until either side typed `bye`.

Experiment 11: Implementing Our Own Ping Program

Aim

To write a ping program that sends ICMP echo requests using a raw socket and displays the round-trip time of each reply.

Algorithm

1. Read the destination host name and resolve it to an IP address.
2. Create a raw socket with protocol ICMP (needs administrator/root rights).
3. Build an ICMP Echo Request: type 8, code 0, checksum, identifier, sequence number, and payload (a timestamp).
4. Compute the Internet checksum over the whole ICMP message.
5. Send the packet, note the send time, and wait for a reply with a timeout.
6. On a reply, skip the 20-byte IP header, read the ICMP header; if type is 0 (Echo Reply) and the identifier matches, compute RTT = receive time - send time.

7. Repeat for the required count, then print packets sent, received, loss percentage and min/avg/max RTT.

Program (Python 3, run with `sudo` on Linux or as Administrator on Windows)

```
import socket, struct, time, os, sys

ICMP_ECHO_REQUEST = 8

def checksum(data):
    s = 0
    if len(data) % 2:
        data += b"\x00"
    for i in range(0, len(data), 2):
        s += (data[i] << 8) + data[i + 1]
    s = (s >> 16) + (s & 0xFFFF)
    s += s >> 16
    return ~s & 0xFFFF

def build_packet(ident, seq):
    header = struct.pack("!BBHHH", ICMP_ECHO_REQUEST, 0, 0, ident, seq)
    payload = struct.pack("!d", time.time())
    chk = checksum(header + payload)
    header = struct.pack("!BBHHH", ICMP_ECHO_REQUEST, 0, chk, ident, seq)
    return header + payload

def ping_once(sock, dest, ident, seq, timeout=2):
    sock.sendto(build_packet(ident, seq), (dest, 0))
    start = time.time()
    sock.settimeout(timeout)
    try:
        while True:
            data, addr = sock.recvfrom(1024)
            end = time.time()
            icmp = data[20:28]                      # skip 20-byte IP header
            t, code, chk, rid, rseq = struct.unpack("!BBHHH", icmp)
            if t == 0 and rid == ident and rseq == seq:
                return (end - start) * 1000, len(data) - 20, data[8]   # rtt,
size, ttl
    except socket.timeout:
        return None

host = sys.argv[1] if len(sys.argv) > 1 else input("Enter host: ")
```

```
count = 4
dest = socket.gethostbyname(host)
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
ident = os.getpid() & 0xFFFF

print(f"PING {host} ({dest}) with {count} requests")
rtts = []
for seq in range(1, count + 1):
    r = ping_once(sock, dest, ident, seq)
    if r:
        rtt, size, ttl = r
        rtts.append(rtt)
        print(f"Reply from {dest}: bytes={size} seq={seq} ttl={ttl} time=
{rtt:.1f} ms")
    else:
        print(f"Request timed out for seq={seq}")
    time.sleep(1)

lost = count - len(rtts)
print(f"\n--- {host} ping statistics ---")
print(f"{count} sent, {len(rtts)} received, {lost * 100 // count}% packet
loss")
if rtts:
    print(f"rtt min/avg/max =
{min(rtts):.1f}/{sum(rtts)/len(rtts):.1f}/{max(rtts):.1f} ms")
sock.close()
```

Output

```
$ sudo python3 myping.py google.com
PING google.com (142.250.195.78) with 4 requests
Reply from 142.250.195.78: bytes=16 seq=1 ttl=117 time=14.3 ms
Reply from 142.250.195.78: bytes=16 seq=2 ttl=117 time=13.9 ms
Reply from 142.250.195.78: bytes=16 seq=3 ttl=117 time=14.1 ms
Reply from 142.250.195.78: bytes=16 seq=4 ttl=117 time=14.0 ms

--- google.com ping statistics ---
4 sent, 4 received, 0% packet loss
rtt min/avg/max = 13.9/14.1/14.3 ms
```

If the program is not run with raw-socket privileges it stops with `PermissionError: [Errno 1] Operation not permitted`. Times and IP will differ on your network.

Result

A ping program was implemented with ICMP echo request and reply messages over a raw socket, and the round-trip time and packet loss were reported.

Experiment 12: Packet Sniffing using Raw Sockets

Aim

To write a program that uses raw sockets to capture packets from the network and display the Ethernet, IP and TCP/UDP/ICMP header details.

Algorithm

1. Create a raw socket: on Linux use `AF_PACKET` with `SOCK_RAW` and protocol `ntohs(3)` (ETH_P_ALL) so that all frames are received.
2. Run in an infinite loop and call `recvfrom` to get each frame.
3. Unpack the first 14 bytes as the Ethernet header: destination MAC, source MAC, EtherType.
4. If EtherType is 0x0800 (IPv4), unpack the IP header: version, header length, TTL, protocol, source IP, destination IP.
5. Based on the protocol number: 1 = ICMP (type, code), 6 = TCP (source port, destination port, sequence, flags), 17 = UDP (source port, destination port, length).
6. Print the fields for each packet. Stop with Ctrl+C.

Program (Python 3, Linux, run with `sudo`)

```
import socket, struct

def mac(b):
    return ":".join(f"{x:02x}" for x in b)

sniffer = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(3))
print("Sniffing started. Press Ctrl+C to stop.\n")
count = 0

try:
    while True:
        raw, _ = sniffer.recvfrom(65535)
        count += 1

        # Ethernet header (14 bytes)
```

```
        dest, src, eth_proto = struct.unpack("!6s6sH", raw[:14])
        print(f"\nPacket {count}")
        print(f"Ethernet : {mac(src)} -> {mac(dest)}
type=0x{eth_proto:04x}")

        if eth_proto == 0x0800:                       # IPv4
            ver_ihl, tos, total_len, ident, flags, ttl, proto, chk, s_ip,
d_ip = \
                struct.unpack("!BBHHHBBH4s4s", raw[14:34])
            ihl = (ver_ihl & 0xF) * 4
            print(f"IPv4     : {socket.inet_ntoa(s_ip)} ->
{socket.inet_ntoa(d_ip)}  ttl={ttl}  proto={proto}  len={total_len}")
            data = raw[14 + ihl:]

            if proto == 1:                            # ICMP
                t, code, _ = struct.unpack("!BBH", data[:4])
                print(f"ICMP     : type={t} code={code}")
            elif proto == 6:                          # TCP
                sp, dp, seq, ack, off_flags = struct.unpack("!HHLLH",
data[:14])
                flags = off_flags & 0x3F
                names = [n for bit, n in ((32, "URG"), (16, "ACK"), (8,
"PSH"), (4, "RST"), (2, "SYN"), (1, "FIN")) if flags & bit]
                print(f"TCP      : port {sp} -> {dp}  seq={seq}  flags=
{','.join(names)}")
            elif proto == 17:                         # UDP
                sp, dp, length, _ = struct.unpack("!HHHH", data[:8])
                print(f"UDP      : port {sp} -> {dp}  length={length}")
except KeyboardInterrupt:
    print(f"\nStopped. {count} packets captured.")
```

This version uses `AF_PACKET`, which exists only on Linux. On Windows a raw socket with `SIO_RCVALL` can sniff IP packets only (no Ethernet header).

Output (sample; generate traffic with `ping google.com` or open a web page)

```
$ sudo python3 sniffer.py
Sniffing started. Press Ctrl+C to stop.

Packet 1
Ethernet : 08:00:27:4e:66:a1 -> 52:54:00:12:35:02  type=0x0800
IPv4     : 192.168.1.10 -> 142.250.195.78  ttl=64  proto=1  len=84
ICMP     : type=8 code=0

Packet 2
Ethernet : 52:54:00:12:35:02 -> 08:00:27:4e:66:a1  type=0x0800
IPv4     : 142.250.195.78 -> 192.168.1.10  ttl=117  proto=1  len=84
ICMP     : type=0 code=0

Packet 3
Ethernet : 08:00:27:4e:66:a1 -> 52:54:00:12:35:02  type=0x0800
IPv4     : 192.168.1.10 -> 142.250.195.78  ttl=64  proto=6  len=60
TCP      : port 49512 -> 443  seq=2871923406  flags=SYN
^C
Stopped. 3 packets captured.
```

Result

A packet sniffer was written with raw sockets; it captured live frames and decoded the Ethernet, IP, ICMP, TCP and UDP headers.

Experiment 13: Analysing Web Servers using Webalizer

Aim

To analyse the access logs of web servers using the Webalizer tool and study the usage statistics it reports.

Algorithm / Procedure

1. Install Webalizer (Ubuntu/Debian): `sudo apt install webalizer`.
2. Locate the web server access log, e.g. Apache `/var/log/apache2/access.log` or Nginx `/var/log/nginx/access.log`. For several servers, collect one log file from each.
3. Edit `/etc/webalizer/webalizer.conf`: set `LogFile` to the log path, `OutputDir` to a web-accessible folder (e.g. `/var/www/html/webalizer`), `HostName` to the server's name, and `LogType clf` (Apache/Nginx common log format).
4. Run Webalizer once per server, using a different output directory for each:

5. Open the generated `index.html` in a browser and study the monthly summary, daily and hourly usage, top URLs, top entry pages, top sites, referrers, user agents, and countries.
6. Compare the reports of the different servers (hits, visits, KBytes, error codes).

Program (commands)

```
sudo apt install webalizer

# Server 1 (Apache)
sudo webalizer -c /etc/webalizer/webalizer.conf \
     -n server1.local -o /var/www/html/webalizer/server1
/var/log/apache2/access.log

# Server 2 (Nginx)
sudo webalizer -c /etc/webalizer/webalizer.conf \
     -n server2.local -o /var/www/html/webalizer/server2
/var/log/nginx/access.log

# View the report
xdg-open /var/www/html/webalizer/server1/index.html
```

Output (sample)

```
Webalizer V2.23-08 (Linux 5.15) English
Hostname: server1.local
Reading log file: /var/log/apache2/access.log
Creating output in /var/www/html/webalizer/server1
  5,236 records in 0.12 seconds
  Saving current history information...

Summary report (sample)
  Total Hits               5236
  Total Files              4871
  Total Pages              1204
  Total Visits              612
  Total KBytes Sent       92418
  Avg Hits per Day          174
  Max Hits per Day          421

Response codes (sample)
```

```
200 OK                  4590
304 Not Modified         402
404 Not Found            198
500 Server Error          46
```

These figures are a sample; paste the numbers and screenshots from your own run. A server with a high 404 count has broken links; a high 500 count points to application faults; the hourly chart shows peak load times.

Result

Webalizer was used to process web server logs and produce HTML reports, and the usage of the different web servers was analysed and compared.